

eda_gym_exercises_dataset_v1.1

November 18, 2025

1 EDA – Gym Exercises Dataset

1.1 Objectif

Vérifier que le fichier `gym_exercises_dataset.csv` est correctement chargé et obtenir une vue d'ensemble :

- dimensions du jeu de données (nombre de lignes et de colonnes) ;
- typage des variables (champs textuels, listes de muscles, attributs techniques) ;
- premières observations sur les valeurs manquantes et la présence éventuelle de doublons ;
- cohérence générale du corpus avant analyse statistique et préparation NLP.

1.2 Interprétation attendue

À partir de cette première inspection, il s'agira de commenter :

- la taille globale du dataset (nombre de lignes / colonnes) ;
- la nature des variables (descriptifs textuels, attributs biomécaniques, catégories d'équipements) ;
- la présence éventuelle de valeurs manquantes ou de champs textuels incomplets ;
- l'existence (ou non) de doublons dans les données brutes ;
- la cohérence générale des premières lignes (valeurs lisibles, descriptions complètes, muscles cohérents).

Cette interprétation sert de point de départ pour juger de la qualité du corpus et préparer les étapes suivantes : nettoyage, normalisation textuelle et structuration pour le traitement NLP du projet TrAIn.me.

1.3 Configuration notebook

1.3.1 Importation des librairies essentielles

```
[1]: import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import kagglehub
import prince

from pathlib import Path
```

```

import sys

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from scipy import stats

# === Localiser automatiquement le package 'themes' en remontant l'arborescence
↳===
root = Path.cwd() # ex : .../train.me/src/notebooks/eda/health_fitness_dataset
themes_root = None

for p in [root, *root.parents]:
    if (p / "themes").is_dir():
        themes_root = p
        break

if themes_root is None:
    raise FileNotFoundError(
        f"Impossible de trouver le dossier 'themes' en partant de {root}. "
        "Vérifie l'arborescence : il doit exister un dossier 'themes/' quelque
↳part au-dessus."
    )

if str(themes_root) not in sys.path:
    sys.path.insert(0, str(themes_root))

print(" Ajouté au PYTHONPATH :", themes_root)
print(" Contenu de", themes_root, ":", [x.name for x in themes_root.iterdir()])

from themes.theme_train_me import set_trainme_theme

```

```

c:\Users\fbac\Desktop\Projets\Dev\GitHub\train.me\.conda\Lib\site-
packages\tqdm\auto.py:21: TqdmWarning: IProgress not found. Please update
jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
    from .autonotebook import tqdm as notebook_tqdm

Ajouté au PYTHONPATH :
c:\Users\fbac\Desktop\Projets\Dev\GitHub\train.me\src\notebooks
Contenu de c:\Users\fbac\Desktop\Projets\Dev\GitHub\train.me\src\notebooks :
['eda', 'model_training', 'preprocessing', 'themes', '__init__.py']

```

Configuration d'affichage et thème TrAIn.me

```

[2]: # Options d'affichage pour les tableaux
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 120)

# =====

```

```
# Thème TrAIIn.me (Seaborn/Matplotlib)
# =====
USE_DARK_THEME = True # bascule possible vers un thème clair si besoin

palette_trainme = set_trainme_theme(
    context="talk",
    font_scale=1.05,
    use_dark=USE_DARK_THEME
)

palette_trainme
```

[2]: ['#00E5FF', '#00BFFF', '#0A84FF', '#1E90FF', '#00FFFF', '#64D8FF']

1.3.2 Activation du thème TrAIIn.me

```
[3]: if USE_DARK_THEME:
    palette_trainme = set_trainme_theme(context="talk", font_scale=1.05,
    ↪ use_dark=True)
else:
    sns.set_style("ticks") # style clair sobre, axes renforcés
    sns.set_context("talk", font_scale=1.05)
    palette_trainme = sns.color_palette("deep") # palette par défaut sobre
    print(" Style activé : 'ticks' → sobre, axes renforcés (présentation pro)")

# Exemple de vérification :
print("Palette active :", palette_trainme.as_hex() if hasattr(palette_trainme,
    ↪ "as_hex") else palette_trainme)
```

Palette active : ['#00E5FF', '#00BFFF', '#0A84FF', '#1E90FF', '#00FFFF', '#64D8FF']

1.3.3 3.4.1. Chargement et inspection initiale

Cette section vise à charger le fichier `gym_exercises_dataset.csv`, vérifier ses dimensions et obtenir un premier aperçu des données brutes. Elle reprend la même démarche que pour les autres EDA du projet : contrôle du chargement, visualisation des premières lignes, statistiques descriptives de base, analyse des valeurs manquantes et des doublons, puis liste complète des colonnes disponibles.

Chargement du dataset `gym_exercises_dataset.csv`

```
[4]: # Téléchargement du dataset depuis Kaggle
dataset_path = kagglehub.dataset_download("rishitmurarka/gym-exercises-dataset")

# Recherche du fichier CSV dans le dossier téléchargé
csv_files = [f for f in os.listdir(dataset_path) if f.endswith(".csv")]
print("Fichiers trouvés :", csv_files)

# Construction du chemin complet vers le CSV
```

```

file_path = os.path.join(dataset_path, csv_files[0])

# Lecture du fichier CSV
df = pd.read_csv(file_path)

# Informations de confirmation
print("Dataset chargé avec succès !")
print(f"Dimensions : {df.shape[0]} lignes et {df.shape[1]} colonnes\n")

```

Downloading from
https://www.kaggle.com/api/v1/datasets/download/rishitmurarka/gym-exercises-dataset?dataset_version_number=1...
 100%| | 48.1k/48.1k [00:00<00:00, 574kB/s]
 Extracting files...
 Fichiers trouvés : ['gym_exercise_dataset.csv', 'stretch_exercise_dataset.csv']
 Dataset chargé avec succès !
 Dimensions : 617 lignes et 17 colonnes

Aperçu des premières lignes

[5]: df.head(10)

[5]:

	Exercise Name	Equipment Variation	Utility
Mechanics	Force \		
0	Neck Flexion	Cable	No Basic or Auxiliary
Isolated	Pull		
1	Neck Flexion	Lever (plate loaded)	No Basic or Auxiliary
Isolated	Pull		
2	Lateral Neck Flexion	Lever (plate loaded)	No Auxiliary
Isolated	Pull		
3	Neck Flexion	Lever (selectorized)	No Basic or Auxiliary
Isolated	Pull		
4	Lateral Neck Flexion	Lever (selectorized)	No Auxiliary
Isolated	Pull		
5	Neck Flexion	Weighted	No Basic or Auxiliary
Isolated	Pull		
6	Lateral Neck Flexion	Weighted	No Auxiliary
Isolated	Pull		
7	Wall Front Neck Bridge	Body Weight	No Basic or Auxiliary
Isolated	Push & Pull		
8	Wall Side Neck Bridge	Body Weight	No Auxiliary
Isolated	Pull		
9	Neck Flexion	Suspended	No Basic or Auxiliary
Isolated	Pull		

Preparation

Execution \

- 0 Sit on bench facing away from middle pulley. P... Move head away from pulley by bending neck for...
- 1 Sit on seat in machine. Position padded lever ... Move head forward by flexing neck until chin t...
- 2 Sit on seat in machine with feet apart . Pos... Move head down to side by laterally flexing ne...
- 3 Sit on seat in machine. Position padded lever ... Move head forward by flexing neck until chin t...
- 4 Sit on seat in machine with feet apart. Positi... Move head down to side by laterally flexing ne...
- 5 Place folded towel on weight plate. Lie supine... Move head up by flexing neck until chin touche...
- 6 Place folded towel on weight plate. Lie on ben... Move head up to side by laterally flexing neck...
- 7 Place small or folded cushioned mat on wall or... Roll down onto forehead until nose touches mat...
- 8 Place small or folded cushioned mat on column ... Push side of head into mat and roll onto top o...
- 9 Place belt or strap around end of suspension t... Increase angle of body by bowing head until ch...

Target_Muscles

Synergist_Muscles \

- | | | |
|---|--|-------|
| 0 | Sternocleidomastoid, | None, |
| 1 | Sternocleidomastoid, | None, |
| 2 | Sternocleidomastoid, Splenius, Erector Spinae, Levator Scapulae, Tr... | |
| 3 | Sternocleidomastoid, | None, |
| 4 | Sternocleidomastoid, Splenius, Erector Spinae, Levator Scapulae, Tr... | |
| 5 | Sternocleidomastoid, | None, |
| 6 | Sternocleidomastoid, Splenius, Erector Spinae, Levator Scapulae, Tr... | |
| 7 | Sternocleidomastoid, Splenius, Trapezius, Upper, Levator Scapulae, ... | |
| 8 | Sternocleidomastoid, Splenius, Trapezius, Upper, Levator Scapulae, ... | |
| 9 | Sternocleidomastoid, Splenius, Trapezius, Upper, Erector Spinae, Ce... | |

Stabilizer_Muscles Antagonist_Muscles

Dynamic_Stabilizer_Muscles Main_muscle \

- | | | |
|-----|---|-----|
| 0 | Rectus Abdominis, Obliques, | NaN |
| NaN | Neck | |
| 1 | Latissimus Dorsi, Deltoid, Posterior, Rhomboid... | NaN |
| NaN | Neck | |
| 2 | Latissimus Dorsi, Pectoralis Major, Sternal, P... | NaN |
| NaN | Neck | |
| 3 | Latissimus Dorsi, Deltoid, Posterior, Rhomboid... | NaN |
| NaN | Neck | |
| 4 | Latissimus Dorsi, Pectoralis Major, Sternal, P... | NaN |

NaN	Neck		
5		Rectus Abdominis, Obliques,	NaN
NaN	Neck		
6		NaN	NaN
NaN	Neck		
7		Rectus Abdominis, Obliques, Quadriceps, Erector Spinae,	
NaN	Neck		
8		Gluteus Medius, Gluteus Minimus,	NaN
NaN	Neck		
9		NaN	NaN
NaN	Neck		

	Difficulty (1-5)	Secondary Muscles	parent_id
0	2	Sternocleidomastoid	NaN
1	2	Sternocleidomastoid	NaN
2	2	Sternocleidomastoid, Levator Scapulae	NaN
3	2	Sternocleidomastoid	NaN
4	2	Sternocleidomastoid, Levator Scapulae	NaN
5	2	Sternocleidomastoid	NaN
6	2	Sternocleidomastoid, Levator Scapulae	NaN
7	3	Sternocleidomastoid, Upper Trapezius	NaN
8	3	Sternocleidomastoid, Upper Trapezius	NaN
9	2	Sternocleidomastoid	NaN

Informations sur le typage, les valeurs manquantes et les doublons

```
[6]: # Typage et complétude globale
df_info = df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 617 entries, 0 to 616
Data columns (total 17 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Exercise Name                        617 non-null    object
1   Equipment                            617 non-null    object
2   Variation                            613 non-null    object
3   Utility                             617 non-null    object
4   Mechanics                           617 non-null    object
5   Force                               617 non-null    object
6   Preparation                          617 non-null    object
7   Execution                           617 non-null    object
8   Target_Muscles                      617 non-null    object
9   Synergist_Muscles                   615 non-null    object
10  Stabilizer_Muscles                   489 non-null    object
11  Antagonist_Muscles                   141 non-null    object
12  Dynamic_Stabilizer_Muscles          265 non-null    object
13  Main_muscle                         617 non-null    object
```

```

14 Difficulty (1-5)          617 non-null    int64
15 Secondary Muscles        616 non-null    object
16 parent_id                161 non-null    float64
dtypes: float64(1), int64(1), object(15)
memory usage: 82.1+ KB

```

1.3.4 Statistiques descriptives des variables numériques

```
[7]: display(df.describe().T)
```

	count	mean	std	min	25%	50%	75%	max
Difficulty (1-5)	617.0	2.737439	0.893215	1.0	2.0	3.0	3.0	5.0
parent_id	161.0	307.248447	158.649536	14.0	181.0	341.0	449.0	615.0

1.3.5 Analyse des valeurs manquantes et doublons

```

[8]: # Comptage des valeurs manquantes par colonne
missing_counts = df.isna().sum().sort_values(ascending=False)

print("\n Valeurs manquantes par colonne (top 20) :")
display(missing_counts.head(20))

# Comptage des doublons ligne à ligne
n_duplicates = df.duplicated().sum()
print(f"\n Nombre de doublons (lignes strictement identiques) : {n_duplicates}")

```

Valeurs manquantes par colonne (top 20) :

Antagonist_Muscles	476
parent_id	456
Dynamic_Stabilizer_Muscles	352
Stabilizer_Muscles	128
Variation	4
Synergist_Muscles	2
Secondary Muscles	1
Difficulty (1-5)	0
Main_muscle	0
Exercise Name	0
Equipment	0
Execution	0
Preparation	0
Force	0
Mechanics	0
Utility	0
Target_Muscles	0

dtype: int64

Nombre de doublons (lignes strictement identiques) : 0

Liste complète des colonnes disponibles

```
[9]: print("\n--- Liste des colonnes du dataset ---")
      print(df.columns.tolist())
```

```
--- Liste des colonnes du dataset ---
['Exercise Name', 'Equipment', 'Variation', 'Utility', 'Mechanics', 'Force',
'Preparation', 'Execution', 'Target_Muscles', 'Synergist_Muscles',
'Stabilizer_Muscles', 'Antagonist_Muscles', 'Dynamic_Stabilizer_Muscles',
'Main_muscle', 'Difficulty (1-5)', 'Secondary Muscles', 'parent_id']
```

1.4 Statistiques descriptives et distribution des variables

1.4.1 Objectif

Obtenir une vue d'ensemble chiffrée des variables numériques du dataset :

- résumer la distribution des variables disponibles (Difficulty (1-5), parent_id) ;
- repérer les ordres de grandeur, la dispersion et les éventuelles valeurs extrêmes ;
- identifier la répartition des niveaux de difficulté des exercices ;
- préparer l'identification d'outliers et la sélection de variables pertinentes pour la suite de l'analyse.

1.4.2 Interprétation attendue

À l'issue de cette section, l'interprétation devra mettre en évidence :

- la distribution des niveaux de difficulté des exercices (répartition sur l'échelle 1-5) ;
- le rôle de parent_id comme identifiant de regroupement plutôt que comme variable métier ;
- la présence éventuelle de valeurs extrêmes ou atypiques sur les variables numériques ;
- les premières tendances sur la structure du corpus (exercices majoritairement faciles, intermédiaires ou difficiles).

Ces éléments guideront la suite de l'EDA, notamment la détection d'anomalies et la préparation des features pour les futurs modèles NLP et de recommandation.

1.4.3 Statistiques descriptives globales

```
[10]: # --- Colonnes numériques disponibles ---
num_cols = df.select_dtypes(include="number").columns.tolist()
print(f"Variables numériques détectées ({len(num_cols)}) : {num_cols}")

# Statistiques descriptives de base
stats_num = df[num_cols].describe().T
display(stats_num)

# Quelques repères métier
```

```

if "Difficulty (1-5)" in df.columns:
    print(f"\nDifficulté moyenne : {df['Difficulty (1-5)'].mean():.2f} / 5")
    print("Répartition des niveaux de difficulté :")
    display(df["Difficulty (1-5)"].value_counts().sort_index())

```

Variables numériques détectées (2) : ['Difficulty (1-5)', 'parent_id']

	count	mean	std	min	25%	50%	75%	max
Difficulty (1-5)	617.0	2.737439	0.893215	1.0	2.0	3.0	3.0	5.0
parent_id	161.0	307.248447	158.649536	14.0	181.0	341.0	449.0	615.0

Difficulté moyenne : 2.74 / 5

Répartition des niveaux de difficulté :

Difficulty (1-5)

```

1    41
2   209
3   254
4    97
5    16

```

Name: count, dtype: int64

1.4.4 Distribution des variables clés

```

[11]: # On garde les variables numériques réellement intéressantes
num_vars = [c for c in num_cols if c != "parent_id"] # parent_id = identifiant_
↳ technique
print(f"Variables numériques représentées : {num_vars}")

n = len(num_vars)
if n == 0:
    print(" Aucune variable numérique pertinente à visualiser.")
else:
    n_cols = 2
    n_rows = int(np.ceil(n / n_cols))

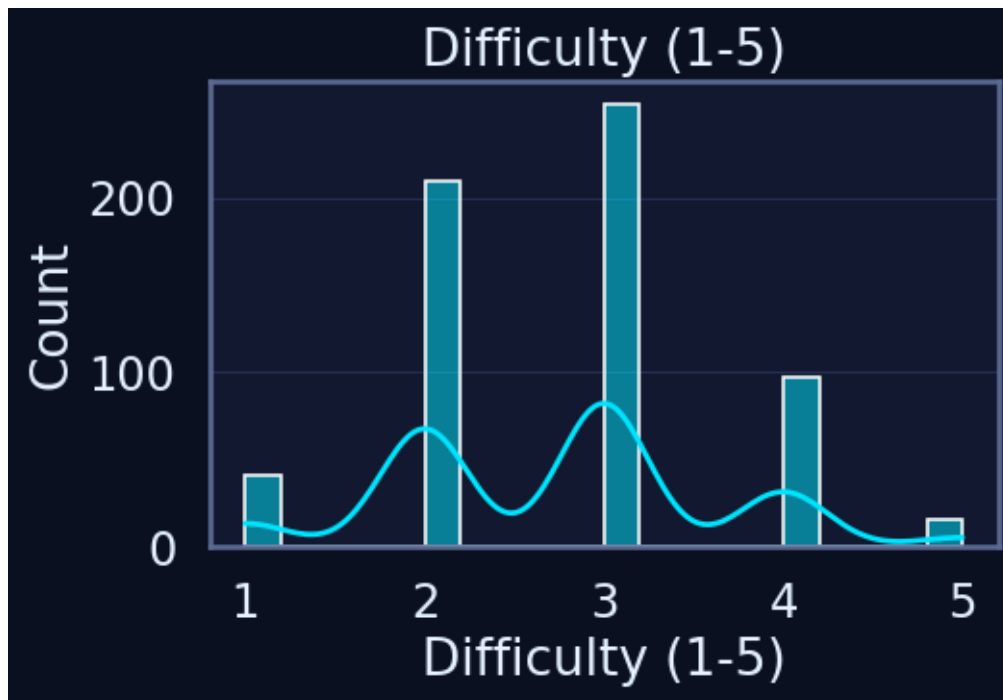
    plt.figure(figsize=(5 * n_cols, 4 * n_rows))

    for i, col in enumerate(num_vars, 1):
        plt.subplot(n_rows, n_cols, i)
        sns.histplot(df[col], bins=20, kde=True)
        plt.title(col)
        plt.grid(True, axis="y")

    plt.tight_layout()
    plt.show()

```

Variables numériques représentées : ['Difficulty (1-5)']

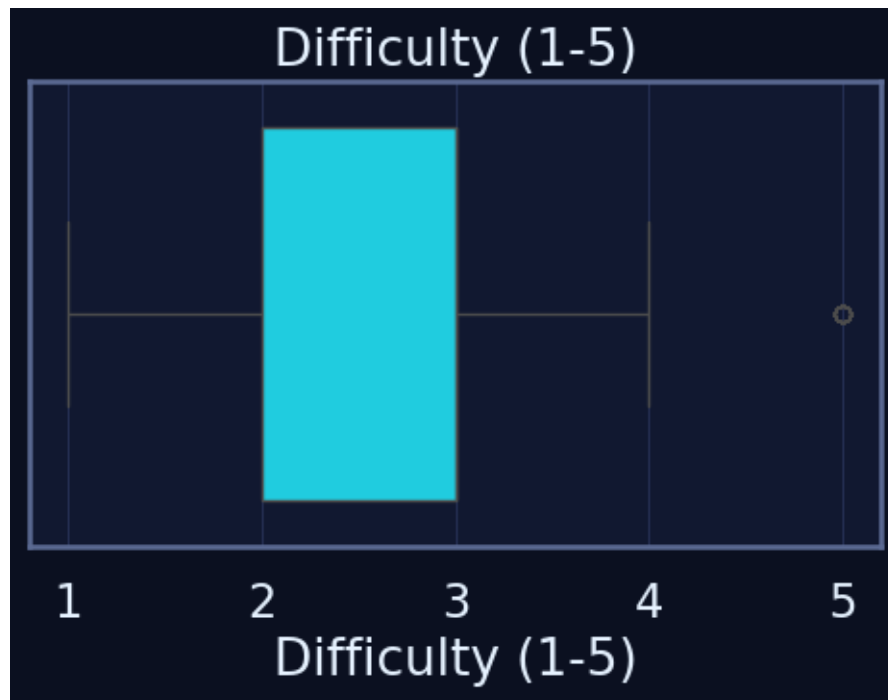


1.4.5 Boxplots pour repérer les outliers

```
[12]: if num_vars:
    plt.figure(figsize=(5 * len(num_vars), 4))

    for i, col in enumerate(num_vars, 1):
        plt.subplot(1, len(num_vars), i)
        sns.boxplot(x=df[col])
        plt.title(col)
        plt.grid(True, axis="x")

    plt.tight_layout()
    plt.show()
else:
    print(" Aucun boxplot pertinent à afficher (pas de variable numérique,
    ↪ métier).")
```



1.4.6 Statistiques descriptives enrichies

```
[13]: if num_cols:
    desc = df[num_cols].describe().T
    desc["missing_%"] = df[num_cols].isna().mean() * 100
    desc["zeros_%"] = (df[num_cols] == 0).mean() * 100
    desc["skew"] = df[num_cols].skew()
    desc["kurt"] = df[num_cols].kurt()

    display(desc.sort_values("std", ascending=False))
else:
    print(" Aucune variable numérique disponible pour des statistiques_
    ↪enrichies.")
```

	count	mean	std	min	25%	50%	75%	↵
↵max missing_%	zeros_%	skew \						
parent_id	161.0	307.248447	158.649536	14.0	181.0	341.0	449.0	615.
↵0 73.905997	0.0	-0.197914						
Difficulty (1-5)	617.0	2.737439	0.893215	1.0	2.0	3.0	3.0	5.
↵0 0.000000	0.0	0.197199						

	kurt
parent_id	-1.036210
Difficulty (1-5)	-0.175658

1.4.7 Détection rapide d'outliers (IQR)

```
[14]: iqr_report = {}

for col in num_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    outliers = df[(df[col] < Q1 - 1.5 * IQR) | (df[col] > Q3 + 1.5 * IQR)]
    iqr_report[col] = len(outliers)

iqr_df = pd.DataFrame.from_dict(iqr_report, orient="index",
    ↪columns=["outliers"])
iqr_df.sort_values("outliers", ascending=False)
```

```
[14]:              outliers
Difficulty (1-5)      16
parent_id              0
```

1.4.8 Aperçu corrélations (sous-ensemble lisible)

```
[15]: if len(num_cols) >= 2:
    subset_cols = num_cols # très peu de variables numériques ici
    corr = df[subset_cols].corr(numeric_only=True)

    plt.figure(figsize=(5, 4))
    sns.heatmap(
        corr,
        annot=True,
        fmt=".2f",
        square=True,
        cbar=True,
        linewidths=0.5,
    )
    plt.title("Corrélations (variables numériques)", fontsize=12, pad=10)
    plt.tight_layout()
    plt.show()
else:
    print(" Corrélation non pertinente : une seule variable numérique ↪
    ↪disponible.")
```

